UNIPAM Educação que transforma	**CENTRO UNIVERSITÁRIO DE PATOS DE MINAS – UNIPAM** **BACHARELADO EM SISTEMAS DE INFORMAÇÃO** **TURMA: 7º PERÍODO** **DESENVOLVIMENTO DE SISTEMAS DE INFORMAÇÃO AVANÇADOS II** **PROFESSOR RAFAEL MARINHO E SILVA**
	NAVEGAÇÃO EM APLICAÇÕES MOBILE

ROTEIRO DE AULA PRÁTICA

Utilize o código fornecido como base para a prática. O objetivo é quebrar a implementação em componentes e arquivos externos, aplicar boas práticas de programação mobile e preparar um repositório no Codespace do GitHub para apresentação ao professor. Apresente um projeto organizado, legível e acessível, que o professor consiga executar diretamente no ambiente do Codespace.

- Extrair o modal para um componente externo, por exemplo components/CustomModal.js.
- Mover todos os estilos para um arquivo separado, por exemplo styles/modalStyles.js, e exportar o objeto styles.

```
import React, { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  Modal,
  TouchableOpacity
} from 'react-native';
// Substituímos o SafeAreaView nativo pelo da biblioteca de contexto para suporte total a notches e
Android
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';

/**
 * Componente de Modal Aprimorado
 * @param {string} animation - Define o tipo de transição (slide, fade, none)
 * @param {string} themeColor - Cor temática para diferenciar visualmente as abas
 */
const CustomModalScreen = ({ animation, themeColor }) => {
  // O estado 'visible' é o booleano que controla a exibição do Modal
  const [visible, setVisible] = useState(false);

  return (
    // SafeAreaView garante que o conteúdo não fique sob a barra de status ou a câmera
    <SafeAreaView style={[styles.screenContainer, { backgroundColor: themeColor + '10' }]}>
      <Text style={[styles.headerText, { color: themeColor }]}>
        Modo: {animation.toUpperCase()}
      </Text>

      {/* Botão principal que altera o estado para true, ativando o Modal */}
      <TouchableOpacity
        style={[styles.mainButton, { backgroundColor: themeColor }]}
        onPress={() => setVisible(true)}
```

```
      >
        <Text style={styles.buttonText}>TESTAR {animation.toUpperCase()}</Text>
      </TouchableOpacity>

      <Modal
        animationType={animation} // Define a física da transição solicitada
        transparent={true}        // Permite visualizar o conteúdo da tela anterior sob o fundo
escurecido
        visible={visible}          // Propriedade booleana controlada pelo useState
        onRequestClose={() => setVisible(false)} // Lida com o botão "voltar" físico do Android
      >
        {/*
            ESTRATÉGIA DO BACKDROP:
            Este TouchableOpacity ocupa 100% da tela. Ao ser tocado, ele fecha o modal.
            O activeOpacity={1} impede que o fundo semi-transparente "pisque" ao ser tocado.
        */}
        <TouchableOpacity
          style={styles.modalOverlay}
          activeOpacity={1}
          onPressOut={() => setVisible(false)}
        >
          {/*
              CARD DO MODAL:
              Importante: Como este View está dentro de um TouchableOpacity, usamos
              uma estrutura que impede que o toque dentro do card feche o modal.
          */}
          <View style={styles.modalCard}>
            {/* Detalhe estético: uma linha superior com a cor do tema da aba */}
            <View style={[styles.colorIndicator, { backgroundColor: themeColor }]} />

            <Text style={styles.modalTitle}>Animação {animation}</Text>

            <Text style={styles.modalBody}>
              {animation === 'slide' && "Perceba como eu subi suavemente do fundo da tela."}
              {animation === 'fade' && "Perceba como eu surgiu alterando a opacidade
(transparência)."}
              {animation === 'none' && "Eu apareci instantaneamente, sem transição suave."}
            </Text>

            {/* Botão de fechamento manual dentro do card */}
            <TouchableOpacity
              style={[styles.closeButton]}
              onPress={() => setVisible(false)}
            >
              <Text style={styles.closeButtonText}>FECHAR</Text>
            </TouchableOpacity>
          </View>
        </TouchableOpacity>
      </Modal>
    </SafeAreaView>
  );
};

// Instanciamos o navegador por abas
const Tab = createBottomTabNavigator();

export default function App() {
  return (
```

```
    // SafeAreaProvider é o contexto obrigatório para que os SafeAreaViews funcionem corretamente
    <SafeAreaProvider>
      <NavigationContainer>
        <Tab.Navigator
          screenOptions={{
            headerShown: false, // Removemos o cabeçalho padrão para um visual mais limpo
            tabBarLabelStyle: { fontSize: 12, fontWeight: 'bold' },
            tabBarActiveTintColor: '#000',
          }}
        >
          {/*
              Definição das três abas. Cada uma passa um animationType diferente
              para o mesmo componente base, permitindo a comparação direta.
          */}
          <Tab.Screen name="SLIDE">
            {() => <CustomModalScreen animation="slide" themeColor="#2196F3" />}
          </Tab.Screen>

          <Tab.Screen name="FADE">
            {() => <CustomModalScreen animation="fade" themeColor="#4CAF50" />}
          </Tab.Screen>

          <Tab.Screen name="NONE">
            {() => <CustomModalScreen animation="none" themeColor="#FF9800" />}
          </Tab.Screen>
        </Tab.Navigator>
      </NavigationContainer>
    </SafeAreaProvider>
  );
}

const styles = StyleSheet.create({
  screenContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  headerText: {
    fontSize: 24,
    fontWeight: '900',
    marginBottom: 20,
  },
  mainButton: {
    paddingVertical: 15,
    paddingHorizontal: 30,
    borderRadius: 12,
    elevation: 4,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.2,
    shadowRadius: 3,
  },
  buttonText: {
    color: '#fff',
    fontSize: 16,
    fontWeight: 'bold',
  },
```

```
  modalOverlay: {
    flex: 1,
    backgroundColor: 'rgba(0, 0, 0, 0.7)', // Cor preta com 70% de transparência para dar foco ao
Modal
    justifyContent: 'center',
    alignItems: 'center',
  },
  modalCard: {
    width: '80%',
    backgroundColor: '#fff',
    borderRadius: 20,
    padding: 25,
    alignItems: 'center',
    overflow: 'hidden', // Necessário para o indicador de cor respeitar o raio da borda
  },
  colorIndicator: {
    width: '120%',
    height: 10,
    position: 'absolute',
    top: 0,
  },
  modalTitle: {
    fontSize: 22,
    fontWeight: 'bold',
    marginTop: 15,
    marginBottom: 10,
    color: '#333',
  },
  modalBody: {
    fontSize: 16,
    textAlign: 'center',
    color: '#666',
    lineHeight: 22,
    marginBottom: 20,
  },
  closeButton: {
    borderWidth: 1,
    borderColor: '#ddd',
    paddingVertical: 10,
    paddingHorizontal: 20,
    borderRadius: 8,
  },
  closeButtonText: {
    color: '#666',
    fontWeight: 'bold',
  },
});
```